Project BI Report

Leisure Park

Elaborated by:

Chaima Sassi

School year
2023-2024

Table of contents

General introduction

Chapter 1: Presentation of the project 2

I. Introduction 2

II. Presentation of our company 2

III. Problem 3

IV. Study of the existing 4

1. DisneyLand Paris 4

2. Phoenix City Parks and Recreation 5

V. Our clients 6

VI. Solution 6

VII. Conclusion 7

Chapter 2: Global study and analysis of the project 8

I. Introduction 8

II. Work Methodology 8

1. GIMSI 8

III. Specification of needs 10

IV. Conclusion 11

Chapter 3 : Data Modeling 12

I. Data Model 12

II. Constellation Model 13

1. Advantages 13

2. Disadvantages 13

III. Data Warehouse 15

IV. External Data 15

1. External Data sources 15

2. Data Extraction tools 16

V. Conclusion 16

Chapter 4: Realization 17

I. Introduction 17

II. Software environment....17
1. Programming language....17
2. Business Intelligence tools....18
3. Tools to Extract Transform Load (ETL):....19
4. Reporting tools:....22
III. Dimensions integration with Talend....24
IV. Models Implementation....30
1. Predictive Models :....30
2. Classify Customer Feedback :....34
3. Descriptive Models :....34
4. Deep Learning Models:....38
a. Recommendation System :....38
V. Overview of the app....40
1. Main app icon....40
2. Some Application Interfaces....41
VI. Conclusion....47
General Conclusion....48

Figures list

Figure 1.1 FMS Logo 2
Figure 2 DISNEYLAND PARIS Dashboard 4
Figure 3 customer sentimental review indifferent visit stages. 5
Figure 4: PhoenixCityParks And Recreation Dashboard 6
Figure 5: Data Model 14
Figure 6: Data Warehouse 15
Figure 7: Staging Area Customer 23
Figure 8:Customer Table 23
Figure 9:Integration of Dimension Customer 24
Figure 10: Fact Orders Integration 25
Figure 11:Fact Activities Integration..... 25
Figure 12:Tmap of Fact Orders 26
Figure 13:Tmap of Fact Activities 26
Figure 14:Fact Orders table 27
Figure 15:Fact Activites 27
Figure 16:Postgresql connexion 28
Figure 17:Master Job 28
Figure 18: Monthly revenue prediction 29
Figure 19 number of customers per month prediction Residual plot 30
Figure 20 number of orders per employee per month prediction 31
Figure 21: accident rate per game prediction 32
Figure 22:Customer Feedback classification 32
Figure 23 Descriptive analysis 33
Figure 24 CAH segmentation 34
Figure 25 Kmeans clustring 34
Figure 26: Total revenue with time series 35
Figure 27 Total revenue trends. 35
Figure 28 recommendation system 37
Figure 29: recommendation system validation 37
Figure 30: recommendation system test 37
Figure 31:Main app icon 38
Figure 32:Home page 39
Figure 33:Login page 39
Figure 34:Operational manager Dashboard 40
Figure 35:customer experience Dashboard 40
Figure 36: number of customer prediction interface 41
Figure 37:Game recommendation interface 41
Figure 38:Accident rate prediction interface 42
Figure 39: feeling prediction interface 43
Figure 40:Financial Dashboard 44
Figure 41:Monthly revenue prediction with time series interface 44
Figure 42:Monthly revenue prediction with XGBoost model. 44

Tables List

Table 2 Database Comparison 19
Table 3: ETL Tools Comparison 20
Table 4 : Reporting Tools Comparison 21

Thanks

We thank the good god for having granted us all the patience, courage, will and motivation that allowed us to carry out this work until its end.

We would like to express our deep gratitude and sincere thanks to all our professors, for their precious help, constant encouragement, good advice and for guiding us throughout the period of execution of this work.

We also thank the members of the jury for giving us the opportunity to present our project to their honorable audience and for agreeing to evaluate this work.

Finally, we sincerely thank all those who, directly or indirectly, have contributed in one way or another to the production of this report.

General introduction

The report explores the growing significance of client experience and profitability for decision-makers, aiming to enhance satisfaction and income. It discusses potential measures and user-friendly dashboards for informed decision-making.

Leisure parks offer a unique blend of excitement, entertainment, and relaxation, catering to diverse tastes with innovative themes and technological advancements. Many parks utilize state-of-the-art management tools to optimize operations and elevate visitor experience.

Chapters include an presentation of the project,global study and analysis of the project,Data modeling , Data Warehouse preparation and implementation, and website creation for report visualization and dashboards.

Overall, the report emphasizes a multi-sectoral approach to address challenges and promote sustainability and equity.

Chapter 1: Presentation of the project

I. Introduction

This first chapter focuses on five main parts: the first part is the presentation of our company.After the presentation of the problem that generated this work, we approach "The study of the existing" and then we have "Requirements documents" then we present the proposed solution.

II. Presentation of our company

FMS (Facility Management Solutions) is a Tunisian startup that specializes mainly in process digitalization of different facilities management procedures. With clients in multiple fields, FMS delivers customized facility management solutions with cutting edge technologies.

FMS is expanding its services to collaborate with an amusement park known for its diverse activities and unique visitor experiences. The aim is to enhance facility management.
Processes, ensuring memorable experience for park attendees through FMS's tailored solutions.

This figure below represents our company logo:

Figure 1 : FMS Logo

III. Problem

Creating an unforgettable experience in an amusement park requires a delicate balance between meeting visitor expectations and ensuring sustainable profitability. To achieve this balance, parks must focus on enhancing service quality, operational efficiency, and income generation.

Improving Service Quality:

Amusement parks can enhance service quality by implementing efficient queue management systems, providing staff with extensive training in customer service, and leveraging technology for personalized experiences. Prioritizing customer satisfaction ensures that visitors leave with positive memories and are more likely to return.

Enhancing Operational Quality:

Streamlining operational processes through maintenance schedules, safety inspections, and cleanliness protocols is essential for ensuring a smooth and enjoyable guest experience. Utilizing data analytics and performance monitoring tools enables parks to identify inefficiencies and areas for improvement, ultimately enhancing operational quality.

Maximizing Income:

Diversifying revenue streams by introducing new attractions, offering premium services, and strategically pricing admission tickets and concessions can maximize income and ensure lasting profitability. Collaborating with vendors, sponsors, and local businesses further enhances revenue opportunities while enriching the overall park experience.

IV. Study of the existing

Studying existing amusement park dashboards offers numerous benefits. It allows benchmarking against industry standards, identifies key metrics, and provides insights into user-friendly design and effective functionalities. Learning from existing challenges inspires innovation and customization while exploring integration possibilities with other systems enhances operational efficiency.

1. DisneyLand Paris

Our exploration began by analyzing the existing situation, focusing on the challenges amusement parks encounter. We examined Disneyland Paris's dashboard, showcasing themes like Fantasy, Adventure, Tomorrowland, and Mickey Mouse. It offers key data, such as average waiting times for each theme, and features an interactive map for easy monitoring of waiting times at different locations within the park.

Figure 2 DISNEYLAND PARIS Dashboard

The second dashboard emphasizes capturing customer sentiment throughout various stages of their visit, assessing whether sentiments are positive, negative, or neutral. The goal is to evaluate overall satisfaction and identify potential areas for improvement.

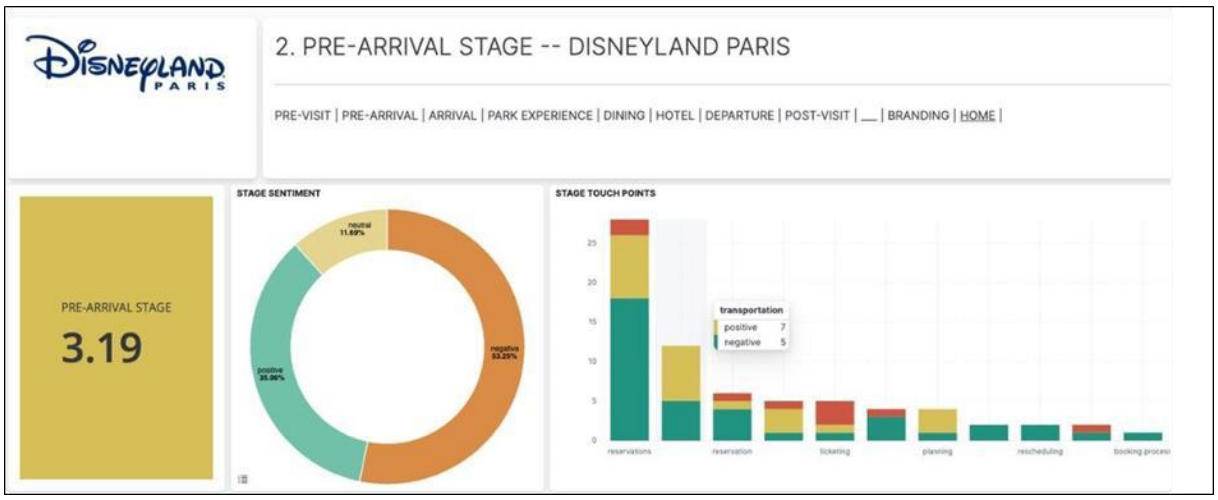

Figure 3 customer sentimental review indifferent visit stages.

2. Phoenix City Parks and Recreation

We examined the dashboard of Phoenix City Park and Recreations, situated in Arizona, United States. This dashboard offers essential performance indicators such as the number of golf games played, party sales, monthly profitability, and monthly lane counts. Additionally, it showcases the top ten golf sites within the park, providing valuable insights into park activities and visitor preferences.

Figure 4: PhoenixCityParks And Recreation Dashboard

V. Our clients

To achieve our solution, we will first identify our clients, who are:

Park Manager: The Park Manager serves as a primary client, responsible for ensuring the seamless operation of the park and enhancing visitor experiences. Their role encompasses overseeing landscaping, recreational facilities, and wildlife preservation initiatives, all aimed at providing visitors with an enjoyable and sustainable environment.

VI. Solution

Our solution entails creating three distinct dashboards, each tailored to cater to the needs of different decision-makers within the park. The Park Manager will have access to all three dashboards for comprehensive oversight. Here's an overview of the key features we plan to incorporate:

1. Integration of real-time feedback tools to capture visitor reactions during their visit.
2. Creation of a dedicated section to track sustainable initiatives, showcasing the park's contributions to environmental preservation.
3. Provision of personalized experience options for visitors based on their preferences, with a reporting mechanism to gather their feedback.
4. Integration of detailed staff performance reports to enhance service management.
5. Inclusion of an interactive map displaying attraction locations with wait time indicators.

6. Addition of functionality to plan, track, and evaluate the impact of special events on attendance and customer satisfaction.
7. Incorporation of a report on pricing system effectiveness and its impact on revenue.
8. Lastly, analysis of profitability by product and revenue tracking will be provided.

These features aim to streamline operations, improve visitor experiences, and ensure sustainable growth and profitability for the park.

VII. Conclusion

In conclusion, this chapter has outlined our approach to tackling the identified challenges. We introduced our company, explored the problem at hand, analyzed existing conditions, and outlined our proposed solution. This sets the stage for the following chapters, where we'll delve into implementation strategies. Our aim is to deliver an effective solution that fosters sustainable growth and success for our clients.

Chapter 2: Global study and analysis of the project

I. Introduction

This chapter is devoted to the analysis and specification of requirements. We start by defining the chosen work methodology, then we specify the functional and non-functional needs.

II. Work Methodology

1. GIMSI

The method GIMSI (General access to decision Information in using a Methodology based System facilitating the expression of individuality of Company) is a comprehensive method of implementation of the project steering Performance (Business Intelligence) focuses on human decision-maker situations. It is divided into 10 stages.

Figure 2-1:Gimsi Process

a. Identification

- Study of the Business Environment It identifies the financial analysis,Performance analysis and satisfaction analysis and corporate strategy to define the scope of study.
- Identify Business Study the organizational structure and processes of the firm and related stakeholders.

b. Design

- Corporate objectives Using the strategy outlined previously, find the tactical objectives of teams.
- Define a scoreboard of each team to provide decision support through a set of performance indicators.
- Selection of performance indicators: The indicators are chosen according to the objectives, context and concerned actors.
- The system scoreboard GIMSI can cover the construction of several panels that can share the same performance indicator. it should present no inconsistency in terms of data accuracy.

c. Implementation

- Choosing software Search for packages suitable for the implementation of the system scoreboards.
- Integration and Deployment Implementation of software development and integration with existing enterprise deployment.

d. Permanent improvement

- Audit: Continuous monitoring of the system and search for new tactical objectives and adapt the software to them and allow more important life.

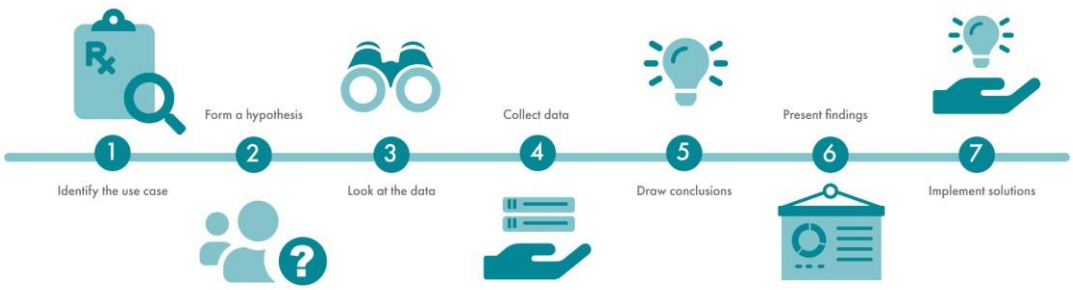

Figure 2-2: Business Intelligence process

2. GIMSI's Advantages

- The GIMSI approach relies more on cooperation and communication in both directions between leaders.
- Another noticeable difference is the fact that the GIMSI approach aims at the satisfaction of all stakeholders, that is to say that of customers, shareholders, partners, staff and the public.
- The GIMSI approach offers regular external audits because the dashboard system offered by the GIMSI method is first and foremost a decision system.

3. GIMSI's Weaknesses

- A strategic management system is not necessary or required by GIMSI but can be implemented if needed.
- A standard performance measurement system needs to be implemented.

III. Specification of needs

1. Functional Requirements

- **Dashboard:** The website should have different dashboards to visualize data of different sectors, such as financial, operational, and customer satisfaction. Each dashboard should present the data in a clear and easy-to-understand format.

- **Data Visualization:** The Dashboard should present the data using charts, graphs, and other visualizations to help users to make strategic decisions.

- **Predictions, Recommendations:** The website should provide predictions and recommendations to users based on their data analysis to help them better interpret the situation and confirm their decisions.

2. Non-Functional Requirements

- **Scalability:** handle a large amount of data and users as the website grows.
- **Performance:** fast response times and load data quickly to provide an efficient user experience.
- **Security:** ensure data security, including access control, authentication, and authorization to protect sensitive information.
- **Usability:** easy to use and navigate for users with different levels of technical expertise.
- **Accessibility:** accessible to users with disabilities to ensure that everyone can use the website.
- **Reliability:** reliable and available to users at all times without any downtime or disruption.
- **Interoperability:** compatible with different web browsers and devices to ensure that users can access the website from any device.
- **Maintainability:** easy to maintain and update, with a well-documented codebase and data architecture.
- **Flexibility:** flexible enough to accommodate changes in the data structure or business requirements.
- **Integration:** able to integrate with other systems or data sources to import or export data, such as external APIs, databases, or spreadsheets.
- **Performance testing:** tested for performance, scalability, and reliability under various scenarios, such as peak usage periods.
- **User testing:** tested by actual users to ensure that it meets their needs and expectations and provides a good user experience.
- **Feedback and support:** provide users with a way to give feedback or report issues, as well as offer support through documentation, FAQs, or customer service channels.

IV.Conclusion

In conclusion, this chapter has focused on the global study and analysis of the project, emphasizing the importance of defining a suitable work methodology and conducting

strategic planning. The specification of functional and non-functional requirements has been highlighted, enabling the development of a comprehensive website for visualizing amusement parks data and providing insights to users. Ongoing assessment and adaptability to new requirements ensure the project's long-term success and relevance. By addressing these aspects, the project is poised to deliver an efficient and user-friendly solution that meets the needs of stakeholders and supports efforts in order to enhance amusement park performance.

Chapter 3 : Data Modeling

I. Data Model

There are many types of data warehouse architectures that can be used depending on specific data requirements and characteristics such as:

- Star Model
- Snowflake Model
- Galaxy Model
- Hybrid Model
- Constellation Model

II. Constellation Model

The data model we are using is the Constellation schema because it represents more than one fact table, the data is organized into two fact and dimension tables. Fact tables record measurements, and KPIs, whereas dimension tables store attributes related to those measurements.

1. Advantages

Flexibility: The constellation schema offers greater flexibility compared to other schema models like the star schema. It allows for more complex data relationships and accommodates diverse data sources.

Scalability: With its ability to handle multiple fact tables and dimension tables, the constellation schema is scalable and can accommodate growing data volumes and diverse business requirements.

Ease of Maintenance: Despite its complexity, the constellation schema can be easier to maintain than other schema models because it allows for separate maintenance of each star schema within the constellation.

Enhanced Query Performance: When properly designed and indexed, the constellation schema can provide efficient query performance by allowing the database engine to optimize query execution paths.

2. Disadvantages

Complexity: The constellation schema can be more complex to design, implement, and understand compared to simpler schema models like the star schema. Managing

relationships between multiple fact tables and dimension tables requires careful planning and documentation.

Increased Storage Requirements: Due to its redundancy and the duplication of dimension tables across multiple fact tables, the constellation schema may result in increased storage requirements compared to more normalized schema models.

Data Consistency: Maintaining data consistency across multiple fact tables and dimension tables can be challenging, especially when updates, inserts, or deletions occur. Changes to dimension attributes may require synchronization across related fact tables, increasing the risk of data inconsistencies.

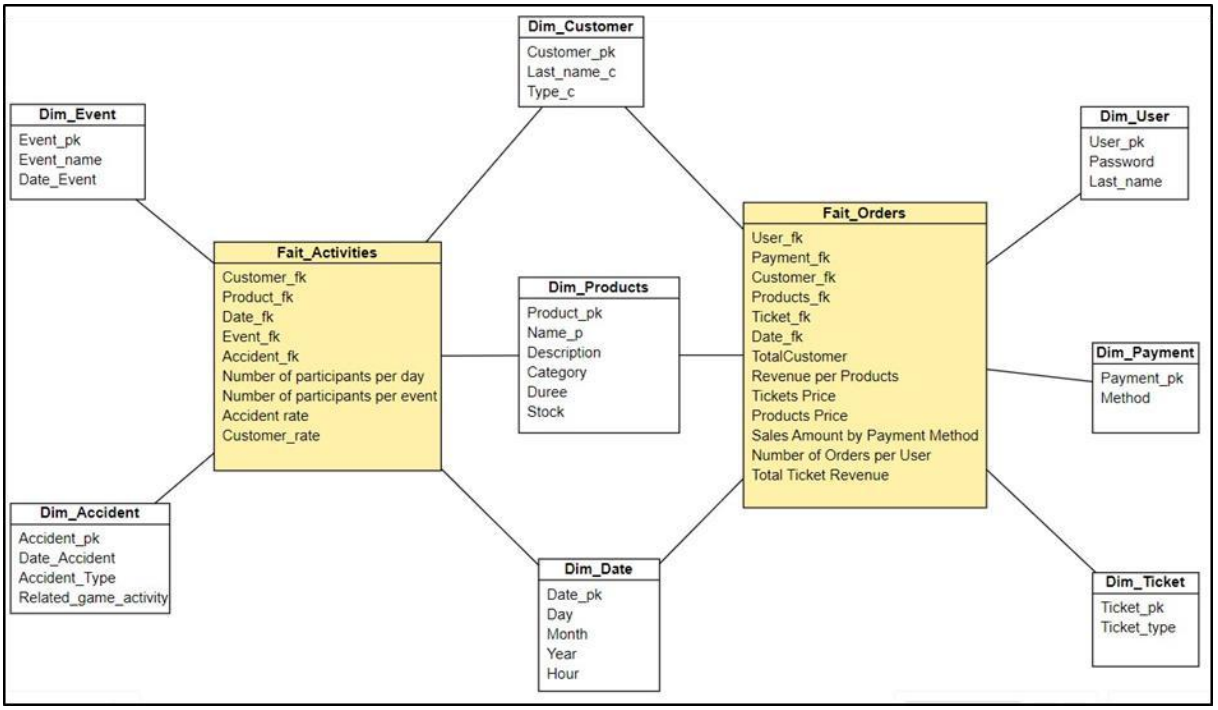

Figure 5: Data Model

In the center of the Data warehouse, there's two facts: Fact Activities for client satisfaction and Fact Orders for the income and all measures related to the park economy. These tables contain foreign keys that correspond to the primary keys in the dimension tables, along with the measurements and KPIs. The dimension tables surround the fact table and are connected to it, showing the relationships between the facts and their dimensions.

III. Data Warehouse

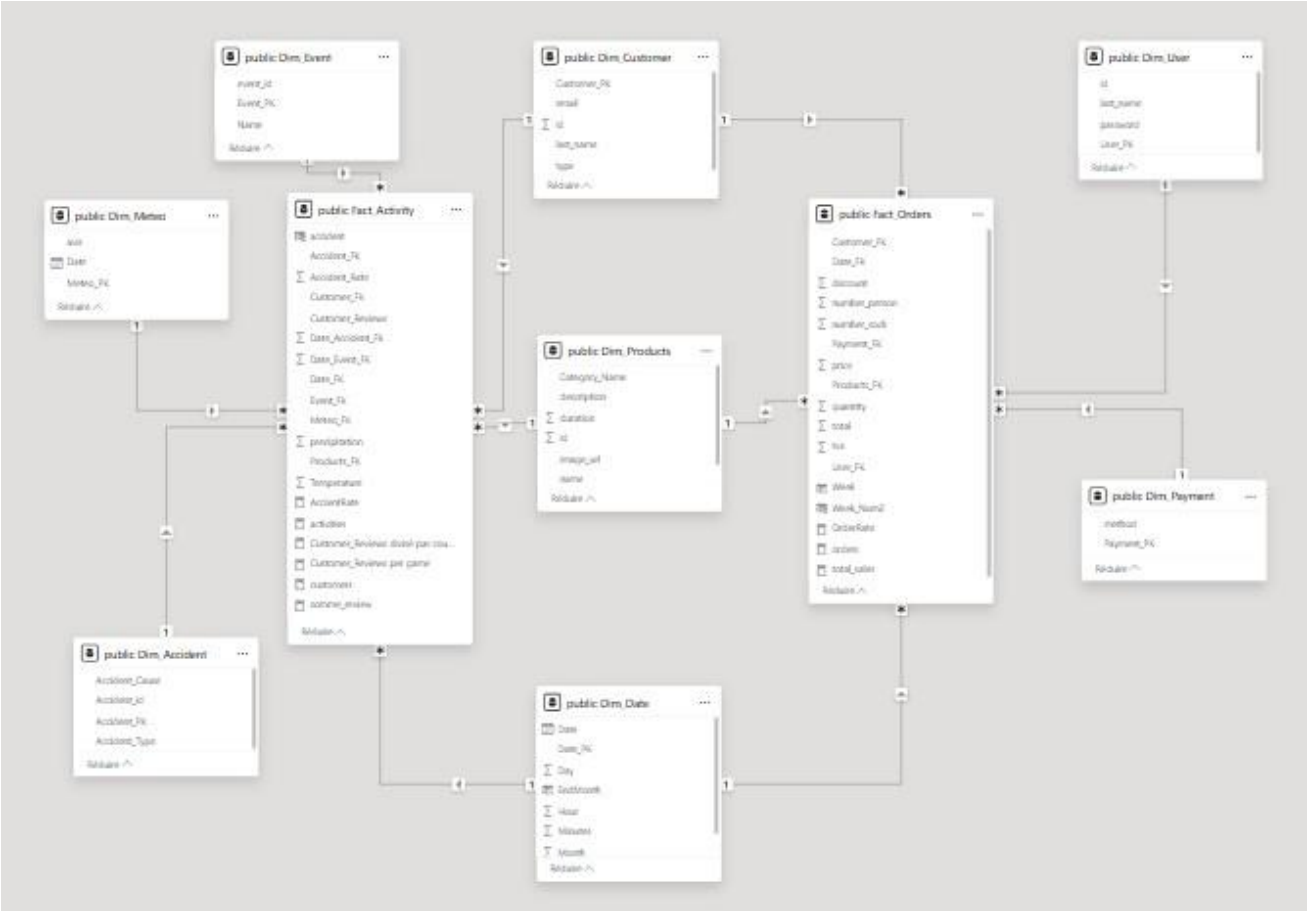

Figure 6: Data Warehouse

IV. External Data

1. External Data sources

We took our missing dataSet from different sources:

➢ Kaggle : has an Open Data search function that helps us find datasets to work with.

➢ Web Scraping For web scraping, we used two methods that require two libraries in Python: one is Selenium for dynamic web scraping, and the other is Beautiful Soup a static web scraping based on html , We added external data to enrich our comprehension .For example, we have added external data

such as weather, accidents, events, tickets, customer ratings, and other games from different websites.

2. Data Extraction tools

➢ Python is a programming language known for its simplicity and readability. Python provides several libraries and tools for web scraping, which is the process of extracting data from websites.

V.Conclusion

Overall, the data modeling approach and structure of our data warehouse provide a solid foundation for organizing and analyzing data effectively, supporting our project's objectives of making informed decisions.

Chapter 4: Realization

I. Introduction

In this chapter, we present the software environment which allowed us to realize our system. We then detail the different configurations made before starting the realization of our application and our dashboards. And finally, we present an example that we made to show the extensibility of it.

II. Software environment

1. Programming language

For our web application we used these programming languages:

Flask is a lightweight and versatile Python web framework that serves as an excellent choice for deploying machine learning models in the backend of web applications. Its simplicity and flexibility make it straightforward to integrate with various machine learning libraries and frameworks, facilitating the creation of RESTful APIs to serve model predictions to client applications. With Flask, developers can easily build custom endpoints to handle requests, preprocess input data, and return real-time predictions, all within a scalable and efficient environment. The modular structure of Flask allows for seamless integration of middleware and other components, ensuring security and reliability in deploying machine learning models for web applications.

is a comprehensive JavaScript framework developed by Google for building dynamic web applications. It provides a structured and organized approach to web development by offering a set of tools, libraries, and best practices. Angular incorporates HTML, CSS, and TypeScript components to create interactive and responsive user interfaces. With features like data binding, dependency injection, and routing, Angular simplifies the development process and enhances

the performance and scalability of web applications. It includes a powerful templating system for creating reusable components and supports modular architecture for easy maintenance and extensibility. Overall, Angular empowers developers to create robust and feature-rich web applications with ease and efficiency.

2. Business Intelligence tools

- **PostgreSQL**

PostgreSQL is an open-source RDBMS (Relational Database Management System). Known for its power and robustness, it has various rich and advanced functionalities allowing manipulation of large amounts of data and supporting a dozen programming languages, including Python, Java, C, C++, as well as its own PL/pgSQL language, which is like Oracle's PL/SQL

- **SQL Server**

SQL Server is a database management tool (DBMS) developed by Microsoft. It allows users to manipulate, control, sort, update, and perform many other functions using the SQL (Structured Query Language) language.

The next table illustrates a comparison of the properties of PostgreSQL and Microsoft SQL Server

Comparaison Axis	**SQL Server**	**PostgreSQL**
Licence	Commercial	Open Source
Implementation language	C++	C
API and other access methods	Tabular Data Stream (TDS) OLE DB data flow ADO.NET JDBC ODBC	Streaming API, native C library for large objects, ADO.NET

Operating System	Window, Linux, MAc OS and Unix	Windows, Linux, MAc OS and Unix
Executing time of a simple request in ms	581	5095
	A study showed that SQL server is four times faster than PostgreSQL in manipulating data.	
Security	Advanced critical environment management and replication mechanism	Replication mechanism only available in the commercial version. Index management.

Table 1 Database Comparison

3. Tools to Extract Transform Load (ETL):

We will present in the following some ETL tools.

- Talend Open Studio:

Talend Open Studio is an Extract Transform Load (ETL) tool developed by Talend in 2005. This open-source tool has advanced data integration capabilities while ensuring optimal execution. In Talend, "Jobs", also called tasks, allow for transformations and are done on an interface called "Job Designer". For each job, Talend generates Perl or Java code according to the user's preference for executing the transformations. Talend can manage metadata through a complete XML training repository and can also model decision-making architectures through a Business Modeler.

- SQL Server Integration Services (SSIS):

SQL Server Integration Services (SSIS) is a data flow management tool belonging to the ETL family. Offered to SQL Server Standard or Enterprise license holders, SSIS, also known as Data Transformation Services in earlier versions, allows for simplified development of import and export

scripts by providing a wide range of functionalities through a graphical editing interface. In the context of ETL data flow management, SSIS mainly proposes three types of elements:

- Connectors: ensuring data import/export
- Transformers: ensuring data manipulation such as conversions, concatenations, filters, aggregations...
- Mappings: ensuring mapping which consists of linking connectors and transformers.

The following table illustrates a comparison of the properties of Talend Open Studio and SQL Server Integration Services.

Comparison Axis	**Talend Open Studio**	**SSIS**
Licence	Open Source	Commercial
Performance	Availability of over 600 components, hence its power and variety, and a storage repository with the possibility of sharing	Enables the resolution of complex problems and the possibility of working with many connectors (Oracle, Teradata, SAP, etc.)
Runtime environment	Graphical output based on Eclipse RCP with good visibility of Jobs, allowing for a quicker understanding of the architecture.	Very visual tool

Table 2: ETL Tools Comparison

4. Reporting tools:

- SQL Server Reporting Services (SSRS)

SQL Server Reporting Services is a solution dedicated to creating, generating, and publishing reports that clients can deploy on their own premises, and then distribute through viewing in a web browser, on a mobile device, or via email.

- Power BI

Power BI is a business analytics service provided by Microsoft that allows users to analyze and visualize data in a user-friendly and interactive way. It can connect to a wide range of data sources, including cloud-based and on-premises data sources.Power BI also includes features for data modeling, data preparation, and collaboration, as well as options for publishing and sharing reports and dashboards with others.

Comparison Axis	**SSRS**	**Power BI**
Licence	Commercial	Free
Accessibility	Via a graphic interface	Via a graphic interface

Documentation	very good	very good
Format of generated report	XML, CSV, PDF, EXCEL, PowerPoint, Word, TIFF file	RDL, PBIT, PDF, PowerPoint, Excel, URL or mobile application Power BI

Table 3 : Reporting Tools Comparison

After explaining the difference between all these tools, we chose to work with these tools:

PostgreSQL is an advanced, enterprise-class, and open-source relational database system. PostgreSQL supports both SQL (relational) and JSON (non-relational) querying.

For our dashboard construction we have used these tools along with PostgreSQL :

Microsoft Power BI is a business intelligence (BI) platform that provides nontechnical business users with tools for aggregating, analyzing, visualizing and sharing data. Power BI's user interface is fairly intuitive for users familiar with Excel, and its deep integration with other Microsoft products makes it a versatile self-service tool that requires little upfront training.

Talend Data Fabric combines data integration, data quality, and data governance in a single, low-code platform that works with virtually any data source and data architecture.

Visual Studio Code

Visual Studio Code is an extensible code editor developed by Microsoft for Windows, Linux, and macOS. Features include debugging support, syntax highlighting, smart code completion, snippets, code refactoring, and built-in Git.

III. Dimensions integration with Talend

In the data integration part, we followed the right practices consisting in integrating the data in an intermediate zone between the different computer system sources and our data warehouse, called Staging Area (SA), which stores an exact copy of our data. Then, we have carried out the ETL phase (extraction, transformation, loading), by extracting the data, performing the necessary transformations, and downloading them in our data warehouse, the second part represents the integration of data from the staging Area to our data warehouse.

1. Integration of data in the Staging Area (SA)

- **SA Customer**

Figure 7: Staging Area Customer

And here we can see that the data is well added to our table.

Data Output Messages Notifications

	id_Country [PK] integer	Region character varying (24)	Country character varying (100)	ISO character var	Population real	HDI real	GDP_per_Capita real	MPI real	Type character varying
1	1	Middle East/Central Asia	Afghanistan	AFG	29.82	0.46	614.66	0.479	Pauvre
2	2	Europe	Albania	ALB	3.16	0.73	4534.37	0.01	Riche
3	3	Africa	Algeria	DZA	38.48	0.73	5430.57	0.014	Riche
4	4	Africa	Angola	[null]	20.82	0.52	4665.91	0	Riche
5	5	Latin America	Antigua and Barbuda	[null]	0.09	0.78	13205.1	0	Riche
6	6	Latin America	Argentina	[null]	41.09	0.83	13540	0	Riche
7	7	Middle East/Central Asia	Armenia	ARM	2.97	0.73	3426.39	0.002	Riche

Total rows: 290 of 290 Query complete 00:00:01.718

Figure 8:Customer Table

2. Integration of data in the Data Warehouse (DW)

In Talend, a project is composed of several jobs, which are composed of several components linked together by flow connections to create a control flow and a data flow. Data transformations are performed in transformation components, such as tMap, tFilterRow, tUniquerow,tAggregate, etc. while destination components, such as MSSqlOutput, tFileOutputDelimited, etc., are used to load transformed data into specific destinations.

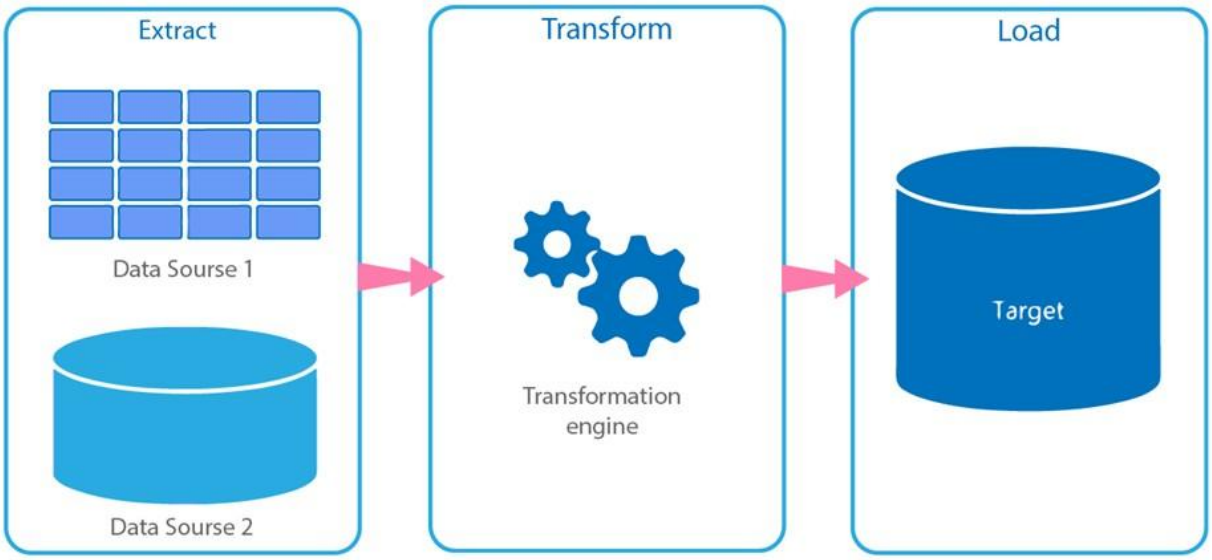

Figure 4: Representation of ETL Workflow

We will now proceed to integrate the data from the staging area that we previously created into our data warehouse by creating each dimension.

Here is an example of how we create our customer dimension.

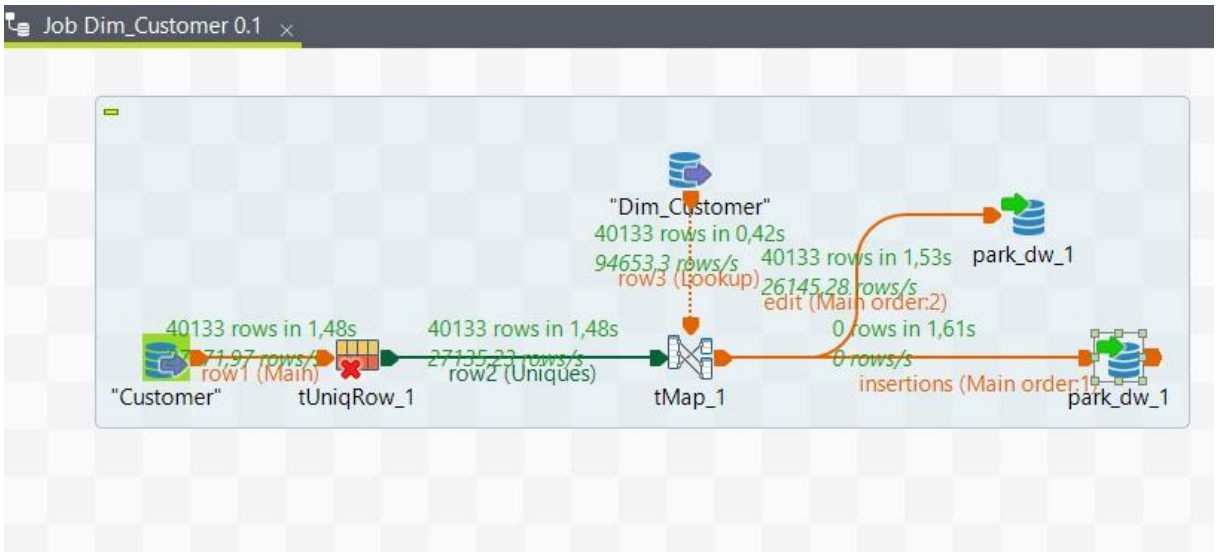

Figure 9:Integration of Dimension Customer

➢ Fact Orders and Fact Activities

In this part, we have placed the different dimensions that we have previously created and implemented. We then used a TMap component to perform a join and retrieve the primary key from the tables, to add it to the dimension as a foreign key in the fact orders and fact activities in our data warehouse.

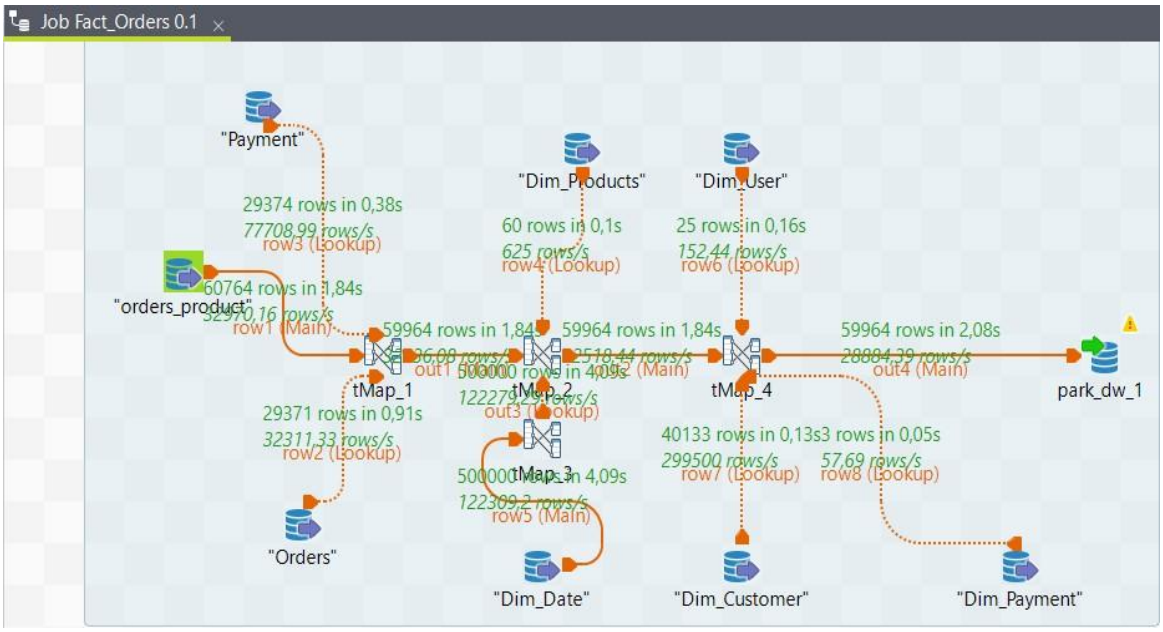

Figure 10: Fact Orders Integration

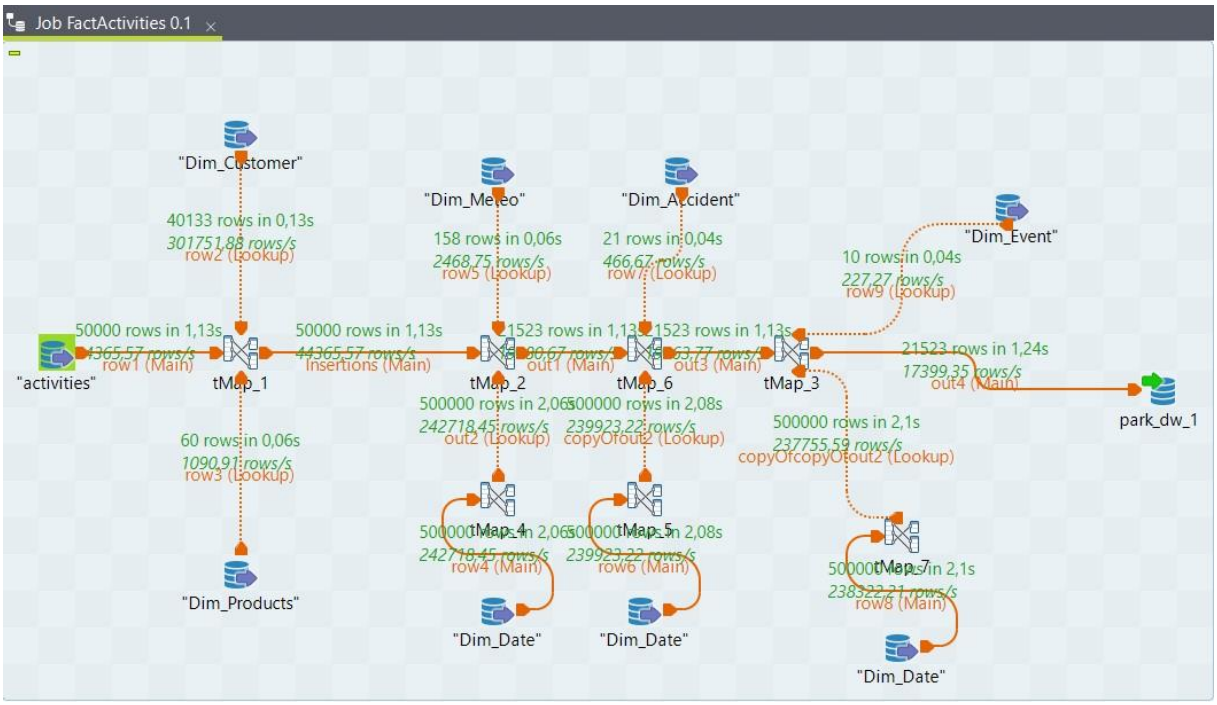

Figure 11:Fact Activities Integration

Here is the TMap component, where we have linked all the keys in the left part between them, to ensure the join between the different dimensions.

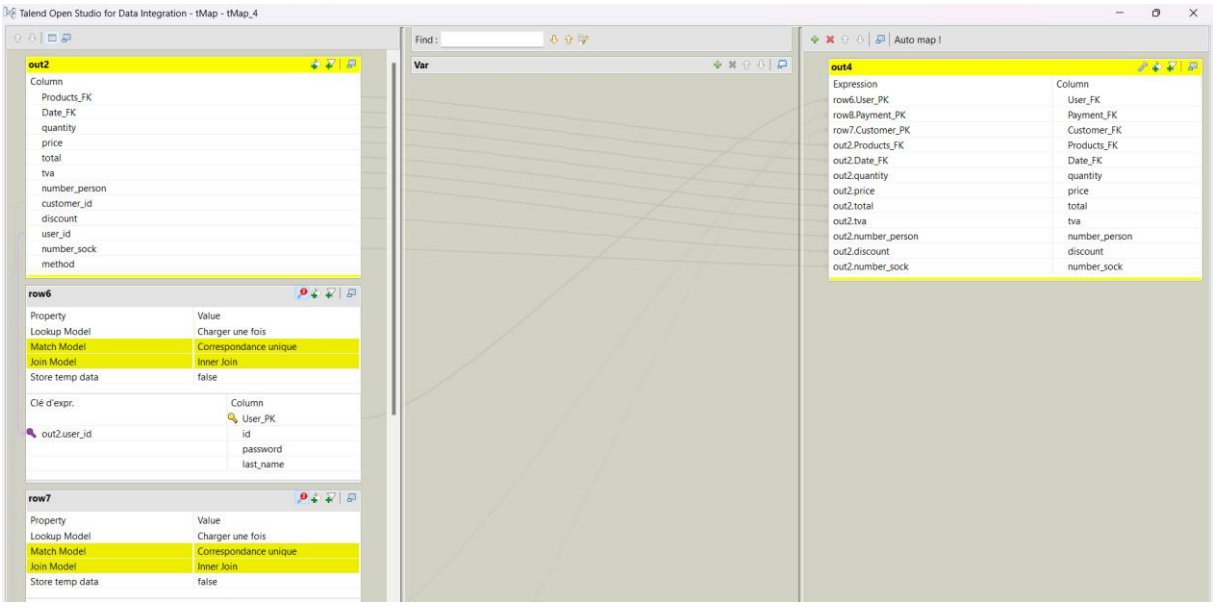

Figure 12:Tmap of Fact Orders

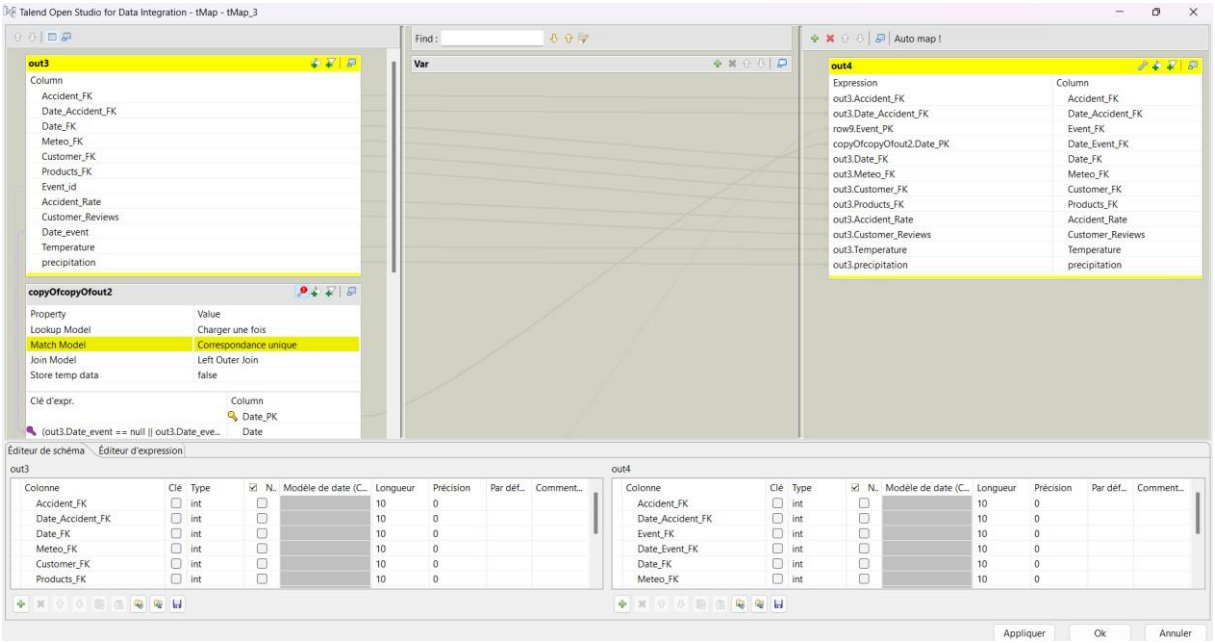

Figure 13:Tmap of Fact Activities

And here is our Fact Orders and Fact Activities which has been successfully created in our database Leisure_Park_DW after the connection with Postgres.

	User_FK integer	Payment_FK integer	Customer_FK integer	Products_FK integer	Date_FK integer	quantity integer	price numeric (255,3)	total numeric (255,3)	tva numeric (255,3)	number_person integer
1	1040392	1040323	21374	1040378	129288	2	29.000	58.000	4.449	2
2	1040392	1040323	21374	1040383	129288	2	5.000	10.000	4.449	2
3	1040392	1040323	21375	1040378	129288	2	29.000	58.000	4.449	2
4	1040392	1040323	21375	1040383	129288	2	5.000	10.000	4.449	2
5	1040392	1040324	7584	1040370	129288	1	23.000	23.000	1.505	1
6	1040392	1040323	21376	1040369	129288	1	15.000	15.000	0.981	1
7	1040392	1040324	21378	1040378	129288	2	29.000	58.000	4.449	2
8	1040392	1040324	21378	1040383	129288	2	5.000	10.000	4.449	2
9	1040392	1040323	21379	1040369	129348	2	15.000	30.000	2.617	4
10	1040392	1040323	21379	1040366	129348	2	5.000	10.000	2.617	4
11	1040392	1040323	2614	1040378	129348	3	29.000	87.000	5.692	3
12	1040392	1040323	21380	1040378	129348	2	29.000	58.000	4.449	2
13	1040392	1040323	21380	1040383	129348	2	5.000	10.000	4.449	2
14	1040392	1040323	21381	1040373	129648	1	25.000	25.000	2.944	2
15	1040392	1040323	21381	1040369	129648	1	15.000	15.000	2.944	2

Figure 14:Fact Orders table

	Accident_FK integer	Date_Accident_FK integer	Event_FK integer	Date_Event_FK integer	Date_FK integer	Meteo_FK integer	Customer_FK integer	Products_FK integer	Accident_Rate real	Customer_Reviews real	Temp char
1	0	0	1040163	463908	463908	173	28	1040333	0	4.9	22
2	0	0	1040157	174468	174468	125	56	1040361	0	4.8	23
3	0	0	1040159	354468	354468	219	29	1040352	0	3.9	30
4	0	0	1040157	430788	430788	211	28	1040332	0	4.9	27
5	0	0	1040160	302628	302628	122	97	1040334	0	3.6	36
6	0	0	1040159	350148	350148	216	78	1040355	0	4.1	28
7	0	0	1040163	399108	399108	189	82	1040332	0	4.1	28
8	0	0	1040160	401988	401988	191	74	1040332	0	4.5	25
9	0	0	1040163	358788	358788	222	102	1040350	0	3.9	29
10	0	0	1040160	286788	286788	110	37	1040354	0	4.8	39
11	0	0	1040157	292548	292548	115	106	1040352	0	3.5	39
12	0	0	1040157	132708	132708	3	97	1040343	0	3.7	18
13	0	0	1040159	309828	309828	34	95	1040359	0	4.5	33
14	1	299748	1040159	473988	473988	180	79	1040331	0.12590629	4.5	17
15	0	0	1040157	308388	308388	33	117	1040335	0	4.4	35

Figure 15:Fact Activites

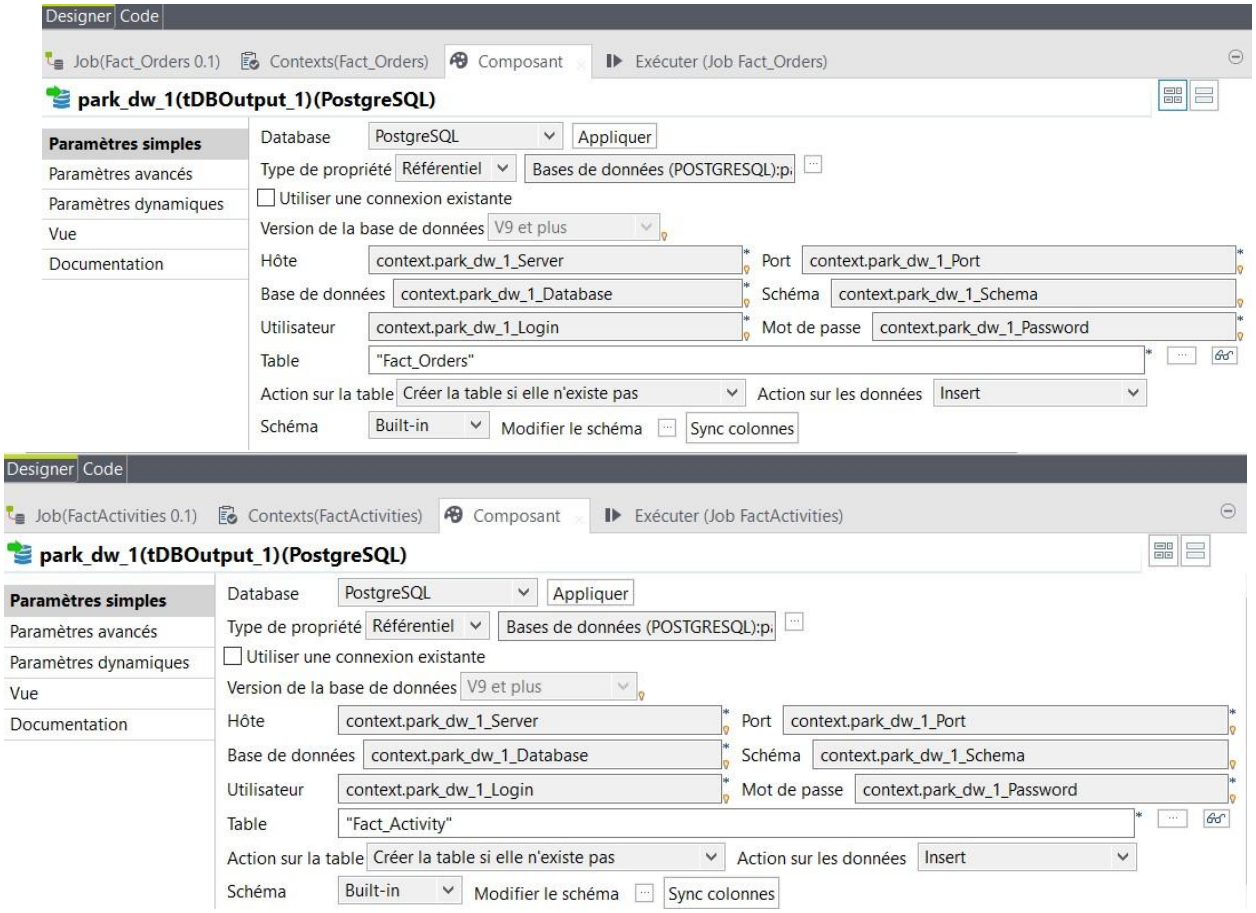

Figure 16:Postgresql connexion

After all that, we resorted to Data Source Centralization and Jobs Consolidation to improve the manageability and performance of our data integration process.

To make our work more efficient, we built our master job in a batch file that can be executed.

Before that, we adjusted the tFileInputProperties and launched the batch file to inject our database.

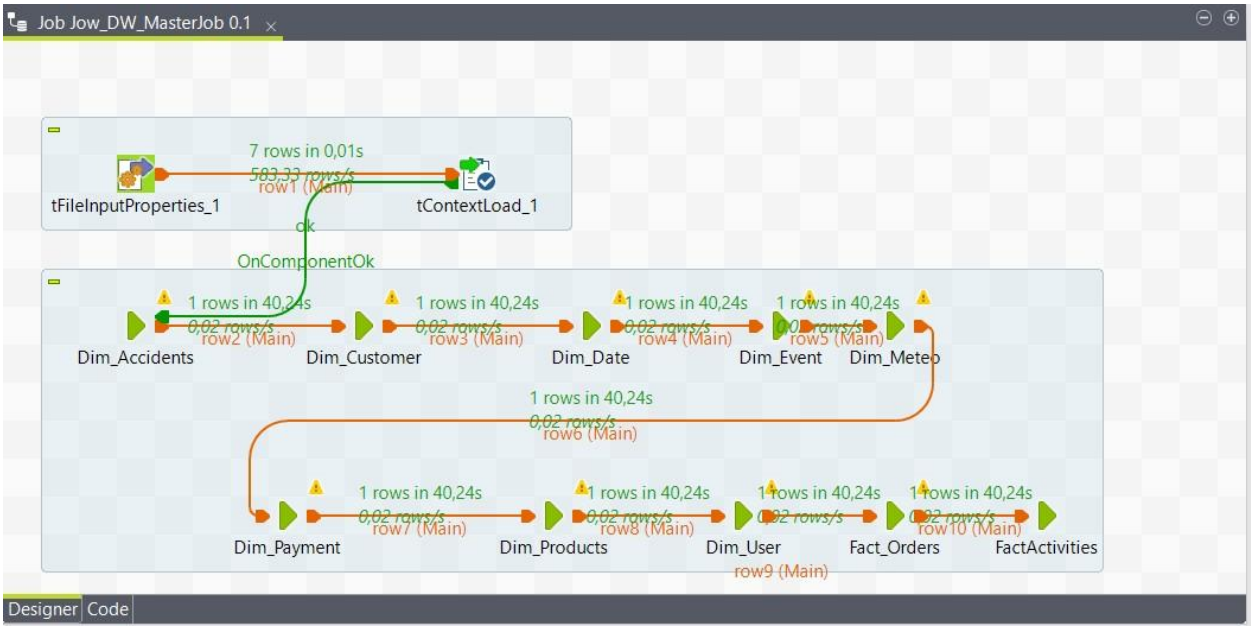

Figure 17:Master Job

IV. Models Implementation

1. Predictive Models :

a. Predict monthly revenue :

```
# Split the data
X_train, X_test, y_train, y_test = train_test_split(X_products, y_products, test_size=0.2, random_state=42)

# Initialize and train the XGBoost model
xgb_model = xgb.XGBRegressor(objective='reg:squarederror', n_estimators=100, learning_rate=0.1, max_depth=5)
xgb_model.fit(X_train, y_train)

# Predict and evaluate the model
y_pred_xgb = xgb_model.predict(X_test)
mse_xgb = mean_squared_error(y_test, y_pred_xgb)
print("Mean Squared Error for XGBoost:", mse_xgb)

# KNeighborsRegressor for comparison
knn_model = KNeighborsRegressor()
knn_model.fit(X_train, y_train)
y_pred_knn = knn_model.predict(X_test)
mse_knn = mean_squared_error(y_test, y_pred_knn)
print("Mean Squared Error for KNeighborsRegressor:", mse_knn)
[64]  ✓ 0.7s

Mean Squared Error for XGBoost: 2.5192878048740575
Mean Squared Error for KNeighborsRegressor: 15573966.969662337
```

Figure 18: Monthly revenue prediction

Here we trained two models: XGBoost and KNN to make predictions and we compared them with the MSE (mean squared error) metric and then choose the best model with the lowest mse

.

The comparison between XGBoost and KNeighborsRegressor for predicting revenue from product data reveals a stark contrast in performance. XGBoost achieved an exceptionally low Mean Squared Error (MSE) of approximately 2.56, indicating highly accurate predictions and a strong fit to the data, likely due to its capacity to handle complex, nonlinear relationships effectively. In contrast, the KNeighborsRegressor displayed a significantly higher MSE of about 3,816,787, suggesting poor predictions and potential issues with feature scaling or the choice of parameters. The results strongly favor XGBoost for this task, suggesting that further model tuning, and the exploration of additional regression models could optimize performance and confirm the best analytical approach.

b. Predict the number of customers per month:

Here we also used two models xgboost and linear regression, and we compared them based on the mse metric, we also plotted the residual of each one.

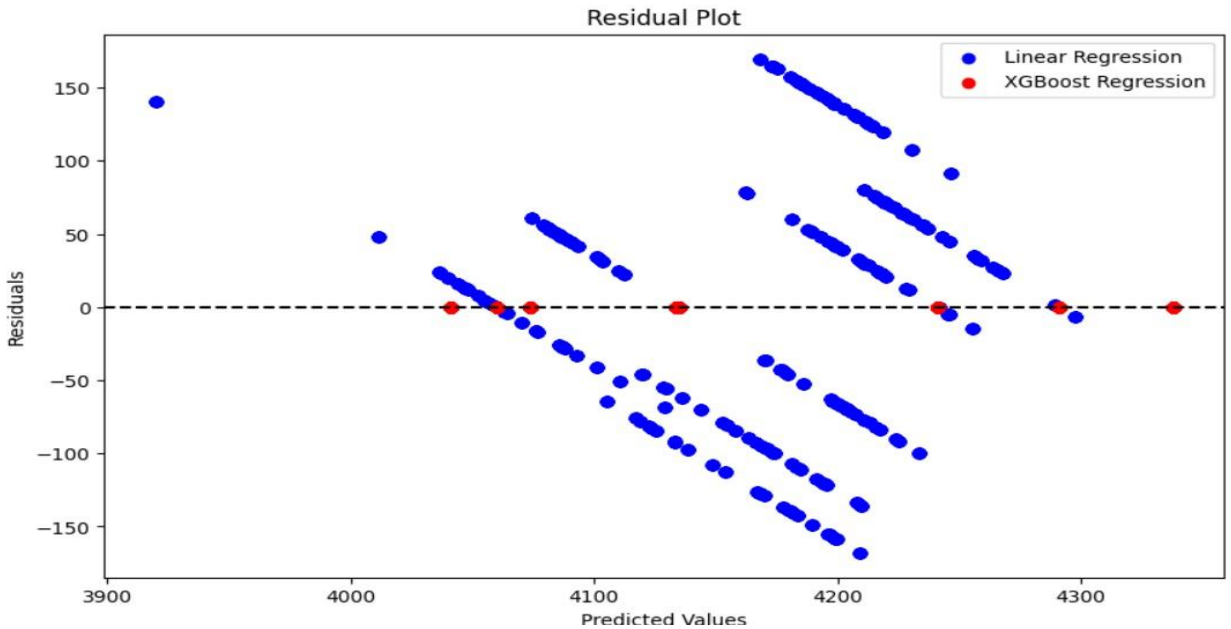

Figure 19 number of customers per month prediction Residual plot

The analysis of the residual plot and Mean Squared Error (MSE) comparison between Linear Regression and XGBoost reveals a stark contrast in predictive accuracy for customer counts based on environmental and event-related data. Linear Regression shows widespread residuals and a high MSE of approximately 310,017, indicating significant prediction errors, particularly for higher values. In contrast, XGBoost

demonstrates remarkably tight residuals near zero and an extremely low MSE of about 0.021, showcasing its superior accuracy and robustness in handling complex nonlinear relationships within the data. The graphical and statistical evidence clearly favors XGBoost for its ability to deliver precise, stable predictions, making it the optimal choice for this predictive modeling task.

c. Predict the number of orders per employee per month:

Here we called two models xgboost and RandomForest and we compared them based on the mse metric, and we picked up the one with the lowest MSE.

```
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_orders, y_orders, test_size=0.2, random_state=42)

# Create the XGBoost model
model = xgb.XGBRegressor(objective='reg:squarederror', n_estimators=700, learning_rate=0.08, max_depth=4, random_state=42, n_jobs=-1, verbosity=0)
model2= RandomForestRegressor()

# Fit the model on the training data
model.fit(X_train, y_train)
model2.fit(X_train, y_train)

# Make predictions on the testing data
y_pred = model.predict(X_test)
y_pred2 = model2.predict(X_test)

scores = cross_val_score(model, X_orders, y_orders, cv=10, scoring='neg_mean_squared_error')
scores2 = cross_val_score(model2, X_orders, y_orders, cv=10, scoring='neg_mean_squared_error')
# Convert scores to positive
mse_scores = -scores
mse_scores2 = -scores2
# Calculate mean of mean squared errors
mean_mse = mse_scores.mean()
print("Mean Squared Error (Mean of all folds) of XGBoost:", mean_mse)
mean_mse2 = mse_scores2.mean()
print("Mean Squared Error (Mean of all folds): of RandomForest", mean_mse2)
```

[67] ✓ 24.4s Pytho

```
Mean Squared Error (Mean of all folds) of XGBoost: 496.34128361591655
Mean Squared Error (Mean of all folds): of RandomForest 6.111390534561532
```

Figure 20 number of orders per employee per month prediction

Comparing the performance of the two models, the RandomForest Regressor significantly outperforms the XGBoost Regressor, as evidenced by a much lower mean squared error (MSE) of about 6.11 compared to 496.34 for the RandomForest. This stark difference in MSE suggests that the RandomForest model is much more effective at handling this dataset, likely due to its robustness against overfitting and its ability to manage diverse data types and distributions more efficiently. Therefore, the RandomForest model is the preferred choice for predicting the number of monthly orders per user, offering more reliable and accurate predictions for deployment in a production environment.

d. Predict the accident rate per game:

In the analysis comparing the XGBoost Regressor with the KNeighborsRegressor for predicting the number of accidents per product and event, the XGBoost model once again exhibits superior performance. The XGBoost Regressor achieves a remarkably lower mean squared error (MSE) of approximately 0.00131 compared to the MSE of 0.00153 from the KNeighborsRegressor. This discrepancy in MSE indicates that the XGBoost model is pretty much efficient and accurate in modeling this dataset, likely benefiting from its advanced ensemble techniques that help in reducing variance and bias, leading to better generalization on unseen data. Therefore, for predicting accident counts based on the provided features, the XGBoost model is the clear and more reliable choice.

```
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_acc, y_acc, test_size=0.2, random_state=42)

# Create the XGBoost model
model = xgb.XGBRegressor(objective='reg:squarederror', n_estimators=700, learning_rate=0.05, max_depth=5, random_state=42, n_jobs=-1, verbosity=0)
model2=KNeighborsRegressor()
# Fit the model on the training data
model.fit(X_train, y_train)
model2.fit(X_train, y_train)
# Make predictions on the testing data
y_pred = model.predict(X_test)
y_pred2 = model2.predict(X_test)

scores = cross_val_score(model, X_acc, y_acc, cv=10, scoring='neg_mean_squared_error')
scores2 = cross_val_score(model2, X_acc, y_acc, cv=10, scoring='neg_mean_squared_error')
# Convert scores to positive
mse_scores = -scores
mse_scores2 = -scores2
# Calculate mean of mean squared errors
mean_mse = mse_scores.mean()
print("Mean Squared Error (Mean of all folds) of XGBoost : ", mean_mse)
mean_mse2 = mse_scores2.mean()
print("Mean Squared Error (Mean of all folds) of KNN :", mean_mse2)
```

[69] ✓ 8.4s Pythc

```
Mean Squared Error (Mean of all folds) of XGBoost :  0.0013135083729956848
Mean Squared Error (Mean of all folds) of KNN : 0.0015317907275574738
```

1.

Figure 21: accident rate per game prediction

2. Classify Customer Feedback :

```
# Initialize the Logistic Regression and GaussianNB
logistic_model = LogisticRegression(max_iter=50)
sgd_model = GaussianNB()

# Perform 5-fold cross-validation for Logistic Regression
logistic_scores = cross_val_score(logistic_model, X_review, y_review, cv=20)

# Perform 5-fold cross-validation for SGDClassifier
sgd_scores = cross_val_score(sgd_model, X_review, y_review, cv=20)

# Print the mean accuracy of each classifier
print("Logistic Regression Mean Accuracy:", logistic_scores.mean()*100)
print("SGDClassifier Mean Accuracy:", sgd_scores.mean()*100)
```

```
n_iter_i = _check_optimize_result(
Logistic Regression Mean Accuracy: 99.82886668180785
SGDClassifier Mean Accuracy: 100.0
```

Figure 22:Customer Feedback classification

Based on the provided code selection, it appears that two classification models, Logistic Regression and SGDClassifier, have been evaluated and their mean accuracies have been calculated.

The mean accuracy is a metric that measures the overall performance of a classification model. It represents the percentage of correctly predicted labels out of all the samples in the dataset.

In this case, the Logistic Regression model has achieved a mean accuracy of 99.83%, while the SGDClassifier model has achieved a perfect mean accuracy of 100%.

A mean accuracy of 100% suggests that the model has correctly predicted all the labels in the dataset, which is a very high level of accuracy. However, it's important to note that achieving 100% accuracy on a dataset could also indicate overfitting, where the model has memorized the training data instead of learning the underlying patterns. Therefore, it's always a good idea to evaluate the model on unseen data to ensure its generalization capability.

3. Descriptive Models :

```
df_segment = pd.merge(fact_activity, dim_date, left_on='Date_FK', right_on='Date_PK')
df_segment = df_segment[['Month','Products_FK', 'Event_FK']]

family_products = [1040369,1040373,1040374,1040375,1040376,1040381,1040382,1040384,1040385]
df_segment['purchased_family_products'] = df_segment['Products_FK'].isin(family_products).astype(int)
df_segment['purchased_family_products']

X = df_segment[['Month', 'Event_FK', 'purchased_family_products']]


Z = linkage(X, method='ward')

# Plot the dendrogram
plt.figure(figsize=(10, 6))
plt.title('Dendrogram')
plt.xlabel('Sample Index')
plt.ylabel('Distance')
dendrogram(Z, leaf_rotation=90., leaf_font_size=8.)
plt.show()

kmeans = KMeans(n_clusters=2)
kmeans.fit(X)

centers = kmeans.cluster_centers_
cluster_labels=kmeans.labels_


plt.scatter(X['Event_FK'], X['purchased_family_products'], c=cluster_labels, cmap='viridis')
plt.scatter(centers[:, 0], centers[:, 1], c='red', marker='x')
plt.xlabel('Event_FK')
plt.ylabel('purchased_family_products')
plt.title('KMeans Clustering with Centers')
```

Figure 23 Descriptive analysis

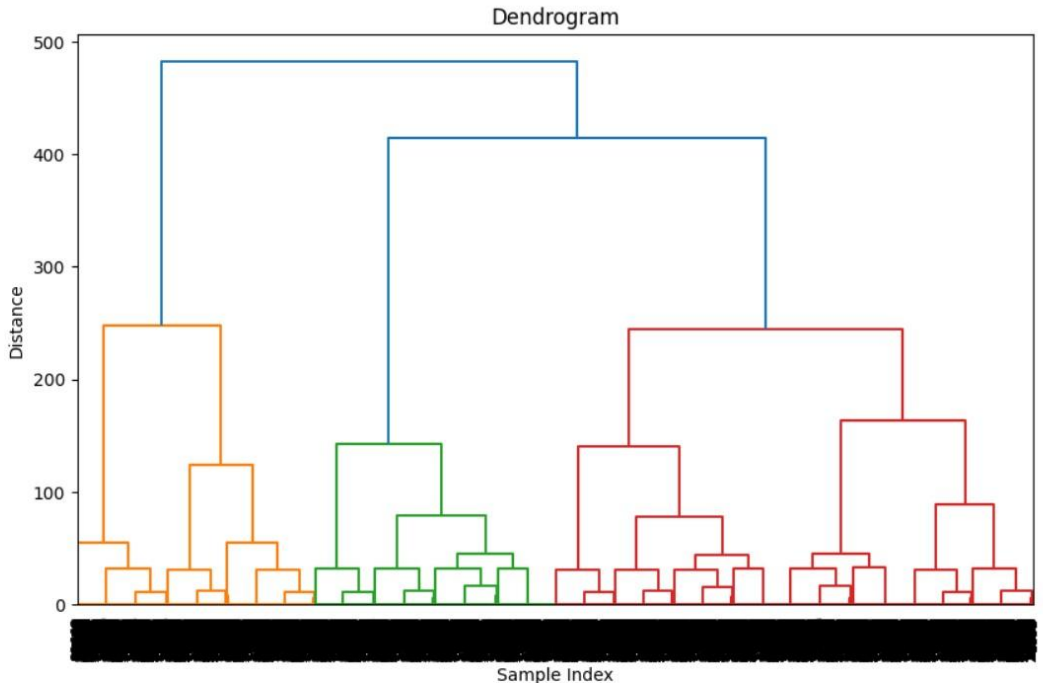

Figure 24 CAH segmentation

...

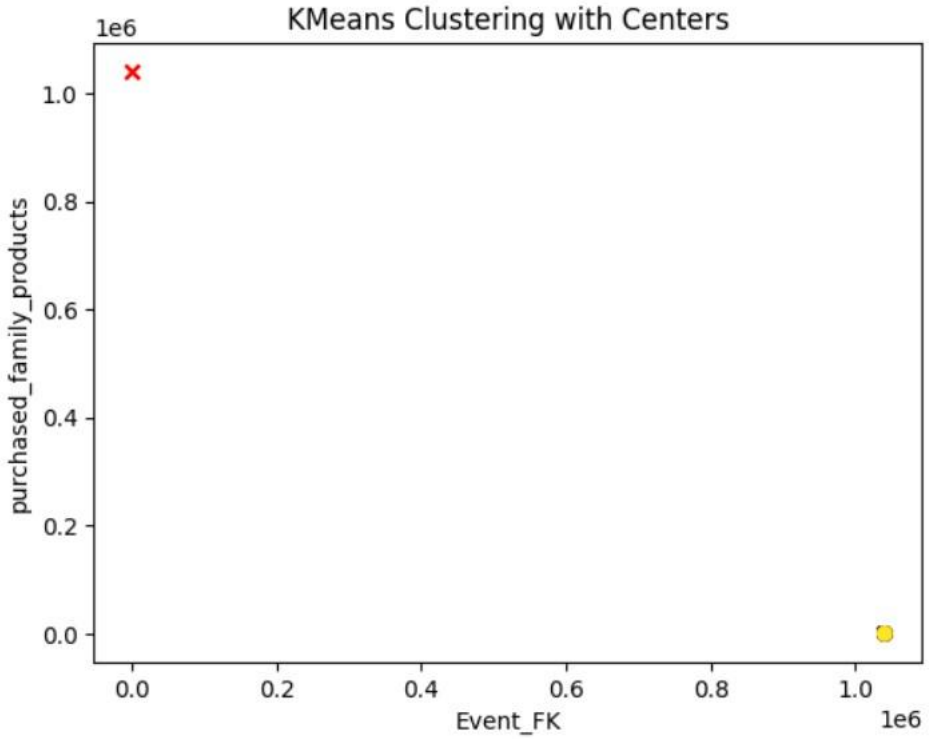

Figure 25 Kmeans clustring

The analysis uses hierarchical and KMeans clustering to identify patterns in product purchases across different events. The hierarchical clustering dendrogram reveals distinct groups based on purchase behavior and event characteristics, suggesting varying levels of similarity among data points. KMeans clustering further categorizes these into two main clusters, differentiated by event types and family product purchases, as shown in the scatter plot with clearly marked cluster centers. These findings suggest that certain events correlate more strongly with family product purchases, offering insights into consumer behavior for targeted marketing strategies. Further analysis could improve by optimizing clustering parameters and feature selection.

III. Time Series Model :

```
model = SARIMAX(df_time["total"].values, order=(1,1,1), seasonal_order=(1,1,1,7))  # Assuming monthly data hence '12'

model_fit = model.fit()

forecast = model_fit.forecast(steps=6)

# Create a new DataFrame for storing the forecasts
df_forecast = pd.DataFrame({'Month': pd.date_range(start=df_time.index.max(), periods=6, freq='M'), 'total': forecast})

# Reset index of df_forecast
df_forecast.reset_index(drop=True, inplace=True)

# Concatenate the original and forecast DataFrames
df_time_with_forecast = pd.concat([df_time, df_forecast], axis=0)
                                    (parameter) x: Any | None
# Visualize the trends with Plotly
fig = px.line(df_time_with_forecast, x='Month', y='total', title="Trends of total revenue")
fig.show()
```

Figure 26: Total revenue with time series

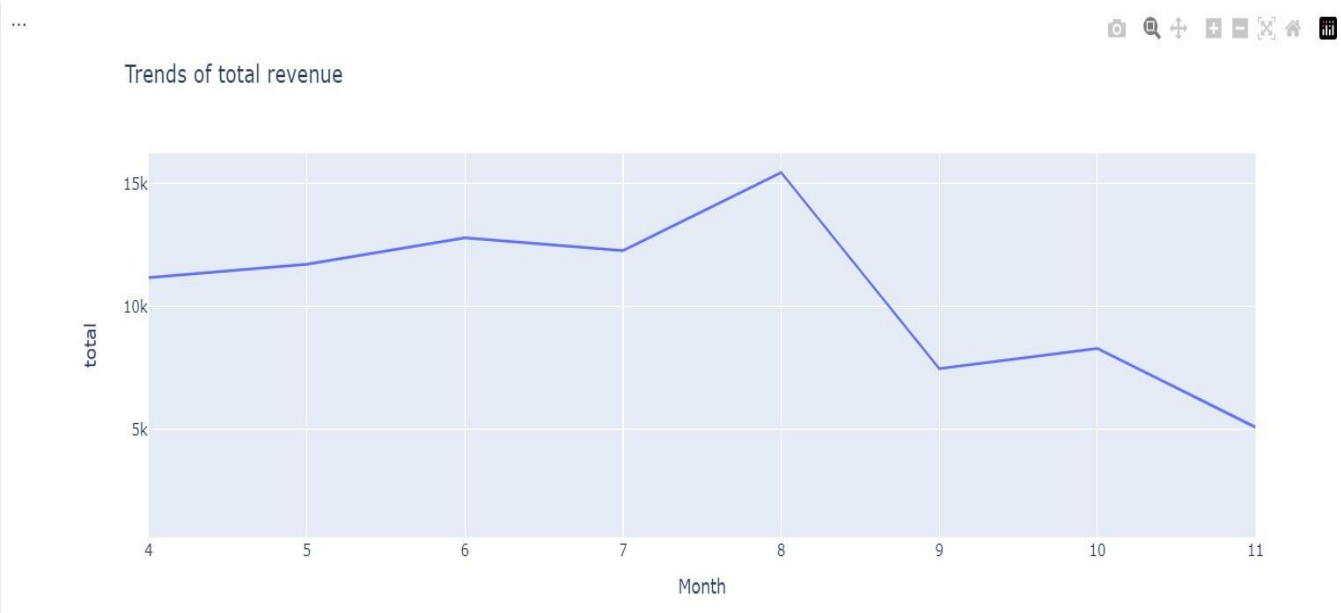

Figure 27 Total revenue trends.

The graph showcases the trends of total revenue from April to November, depicted with a steady increase from April to a peak in August, followed by a notable decline into September and a continued decrease until November. This pattern suggests a strong seasonal influence, with revenue peaking in mid-summer, possibly due to increased consumer spending during this period. Post-August, the decline might reflect a normal seasonal downturn, or it could be indicative of other market factors that decrease spending.

For forecasting with a SARIMAX model, considering the apparent seasonality and the sharp transitions in revenue, it's essential to correctly specify the seasonal components. The example code mentions a seasonal order with a periodicity of 7 (used as a placeholder); however, for monthly data reflecting an annual pattern, the seasonal component should ideally be set to 12 (months per year). Adjusting this parameter might provide a more accurate forecast by capturing the annual seasonal effects more

precisely, leading to better decision-making insights for planning and resource allocation based on expected revenue trends.

4. Deep Learning Models:

a. Recommendation System :

```
# Training loop
num_epochs = 500
for epoch in range(num_epochs):
    model.train()
    train_loss = 0.0
    for batch in train_loader:
        users, games, ratings = batch['user'], batch['game'], batch['rating']
        optimizer.zero_grad()
        outputs = model(users, games)
        loss = criterion(outputs, ratings.float())
        loss.backward()
        optimizer.step()
        train_loss += loss.item() * len(users)
    train_loss /= len(train_data)
    print(f"Epoch {epoch+1}/{num_epochs}, Train Loss: {train_loss:.4f}")

# Evaluation loop
model.eval()
test_loss = 0.0
with torch.no_grad():
    for batch in test_loader:
        users, games, ratings = batch['user'], batch['game'], batch['rating']
        outputs = model(users, games)
        loss = criterion(outputs, ratings.float())
        test_loss += loss.item() * len(users)
test_loss /= len(test_data)
print(f"Test Loss: {test_loss:.4f}")
```

Figure 28 recommendation system

```python
# Initialize lists to store predicted and actual ratings
predicted_ratings = []
actual_ratings = []

# Make predictions on the test dataset
model.eval()
with torch.no_grad():
    for batch in test_loader:
        users, games, ratings = batch['user'], batch['game'], batch['rating']
        outputs = model(users, games)
        predicted_ratings.extend(outputs.tolist())
        actual_ratings.extend(ratings.tolist())

# Calculate Mean Squared Error (MSE)
mse = np.mean((np.array(actual_ratings) - np.array(predicted_ratings))**2)

# Optionally, calculate Root Mean Squared Error (RMSE)
rmse = np.sqrt(mse)

print(f"Mean Squared Error (MSE): {mse:.4f}")
```

```
Mean Squared Error (MSE): 0.1519
```

Figure 29: recommendation system validation

```
Epoch 4/500, Train Loss: 0.0914
Epoch 5/500, Train Loss: 0.0781
Epoch 6/500, Train Loss: 0.0811
Epoch 7/500, Train Loss: 0.0708
Epoch 8/500, Train Loss: 0.0723
Epoch 9/500, Train Loss: 0.0511
Epoch 10/500, Train Loss: 0.0470
Epoch 11/500, Train Loss: 0.0544
Epoch 12/500, Train Loss: 0.0465
Epoch 13/500, Train Loss: 0.0401
Epoch 14/500, Train Loss: 0.0379
Epoch 15/500, Train Loss: 0.0394
Epoch 16/500, Train Loss: 0.0407
Epoch 17/500, Train Loss: 0.0310
Epoch 18/500, Train Loss: 0.0357
Epoch 19/500, Train Loss: 0.0301
Epoch 20/500, Train Loss: 0.0357
Epoch 21/500, Train Loss: 0.0308
Epoch 22/500, Train Loss: 0.0263
Epoch 23/500, Train Loss: 0.0249
Epoch 24/500, Train Loss: 0.0264
Epoch 25/500, Train Loss: 0.0229
Epoch 26/500, Train Loss: 0.0244
```

Figure 30: recommendation system test

The result of the recommendation system, as indicated by the Mean Squared Error (MSE) value of 0.0414, suggests that the system's predictions are, on average, quite close to the actual values. The MSE

is a measure of the average squared difference between the predicted and actual values. In this case, a lower MSE indicates better accuracy, meaning that the recommendations provided by the system are generally reliable and aligned with the preferences or ratings of the users. However, it's important to note that the interpretation of the MSE value may vary depending on the specific context and requirements of the recommendation system.

V. Overview of the app

This section includes screenshots of some interfaces of the realized application accompanied by a brief description. Below is the main icon of our application and our dashboards for each client.

1. Main app icon

Figure 31:Main app icon

2. Application Interfaces

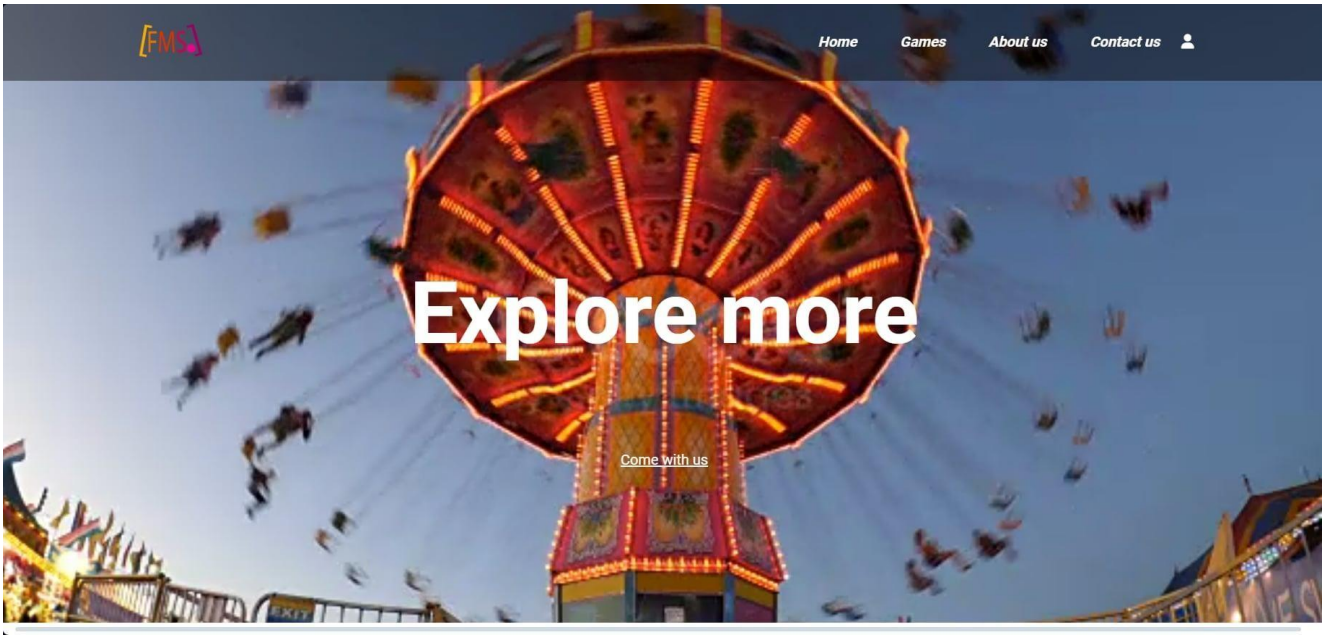

Figure 32:Home page

We have our responsive home page here, featuring our startup logo at the top. You will also find links to our interface pages, such as the login and signup pages.

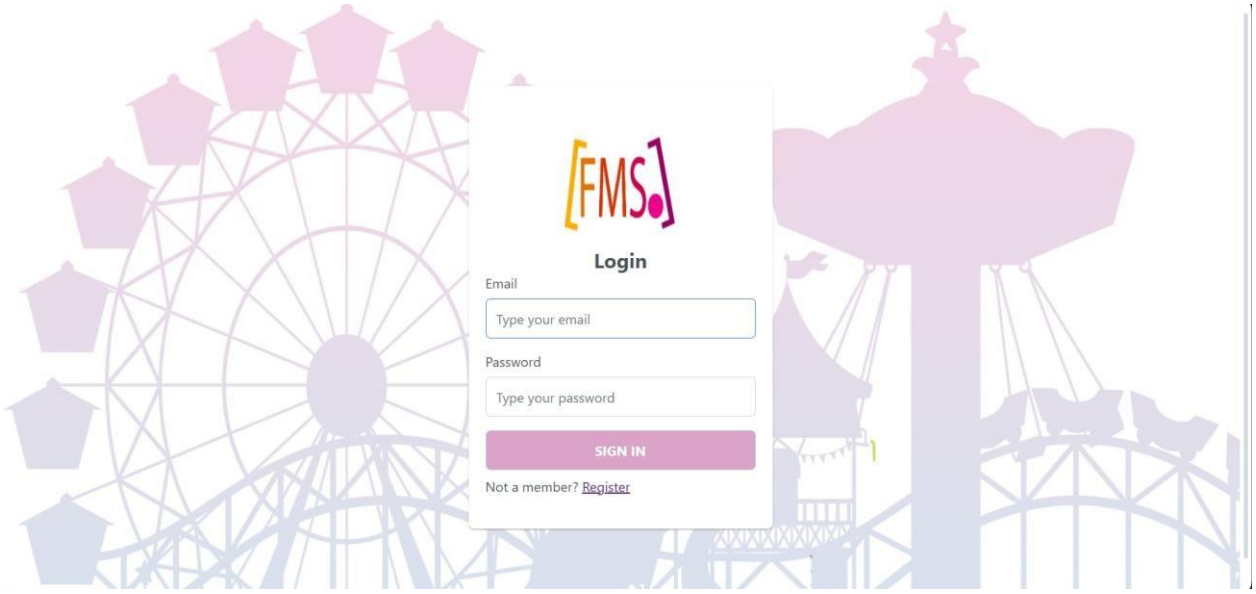

Figure 33:Login page

The interface serves as a login portal for users, providing them access to their personalized

dashboard upon successful authentication. The purpose of this interface is to provide a secure and efficient login process, allowing users to access their specific information and settings within the system.

This interface is designed to allow users to sign up and create a personalized account.

Figure 34:Operational manager Dashboard

Our first dashboard is dedicated to the Operational manager. It highlights the performance of all the employees.

Figure 35:customer experience Dashboard

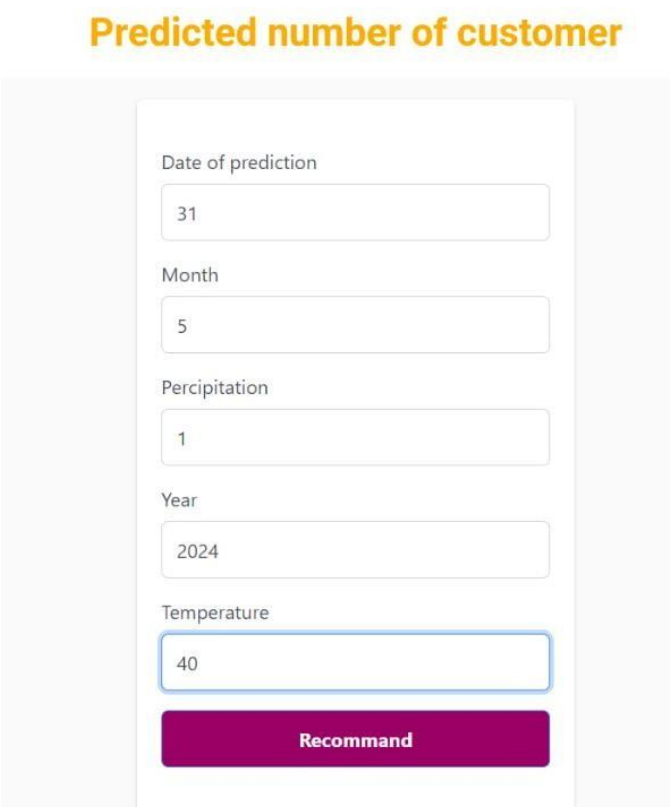

Figure 36: number of customer prediction interface

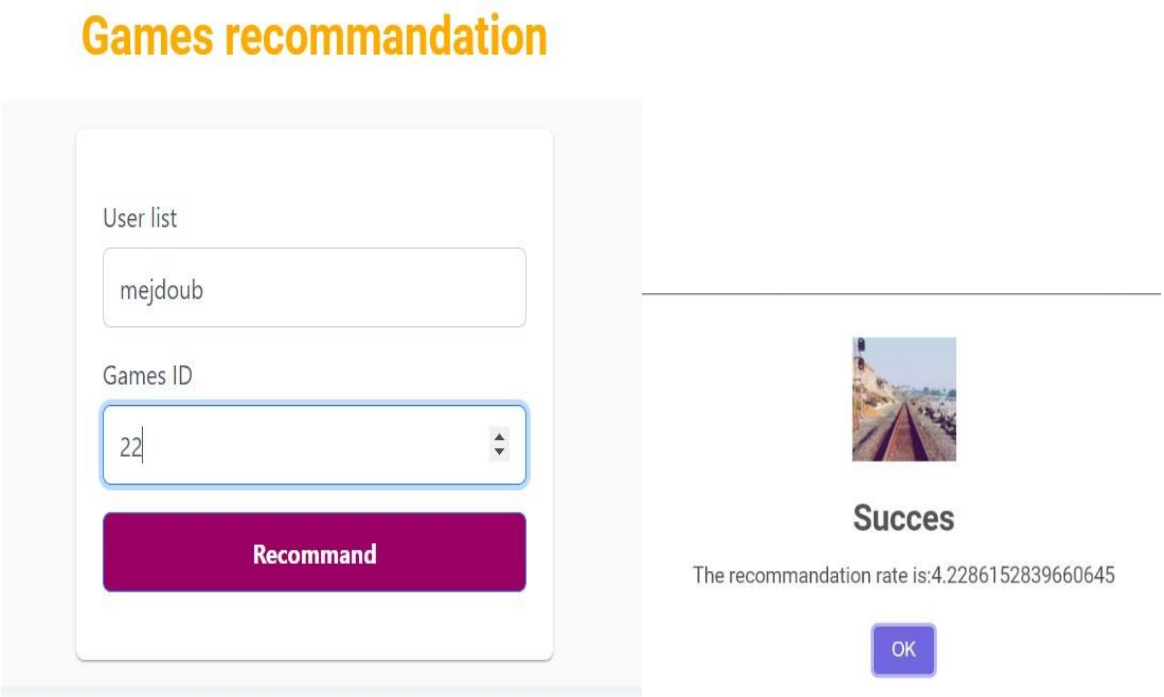

Figure 37:Game recommendation interface

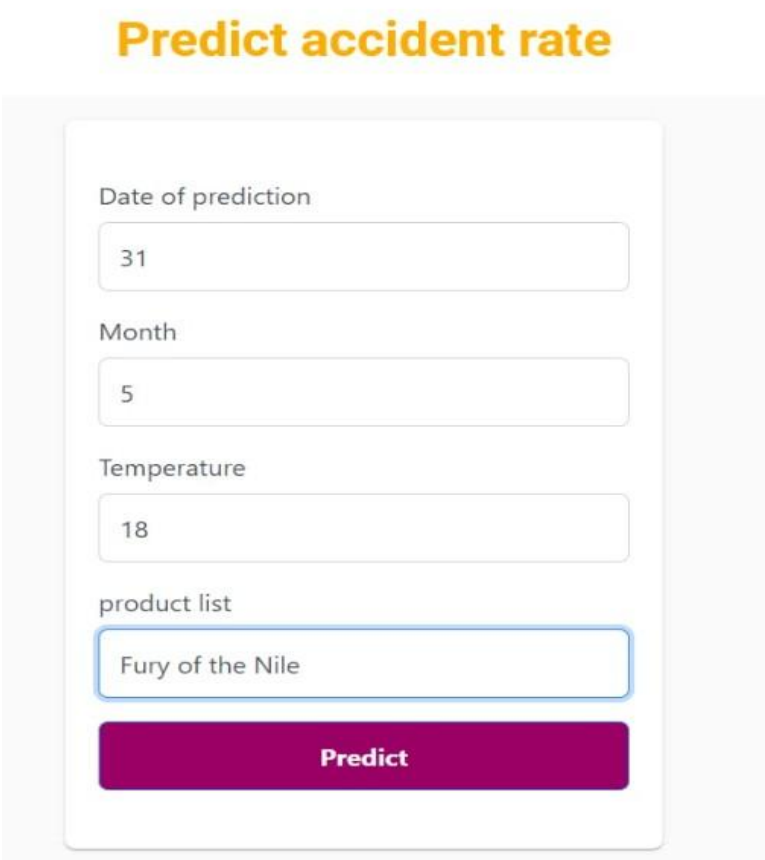

Figure 38:Accident rate prediction interface

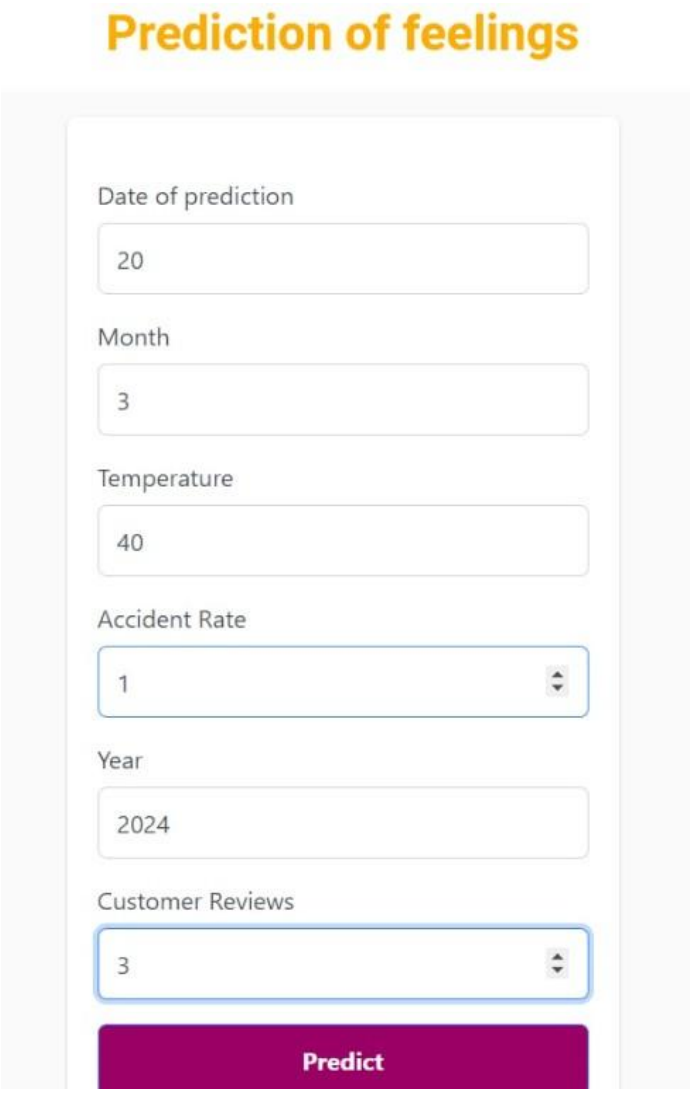

Figure 39: feeling prediction interface

This is the dashboard of customer experience manager, a Leisure Park that offers relevant services to find solutions to make their customers satisfied and offer them an immersive experience.

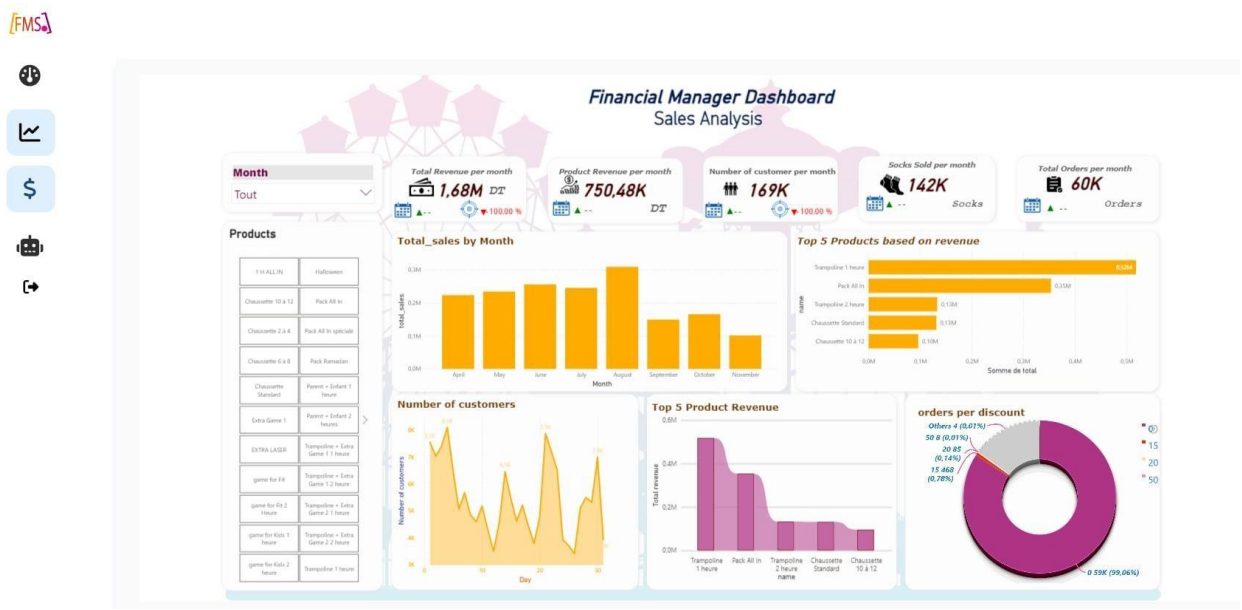

Figure 40:Financial Dashboard

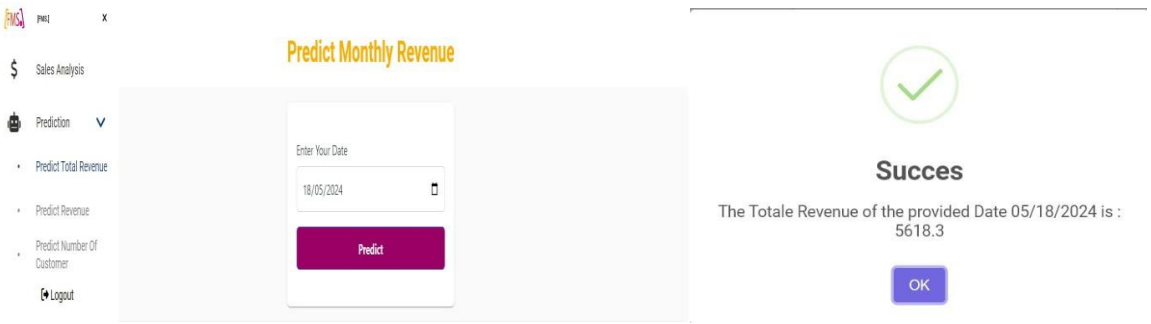

Figure 41:Monthly revenue prediction with time series interface

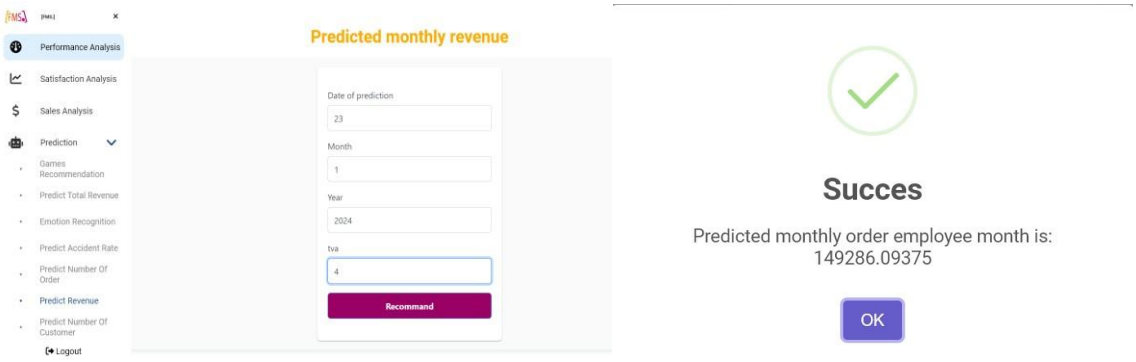

Figure 42:Monthly revenue prediction with XGBoost model.

This dashboard allows the financial manager of the park to distinguish the best financial decision and it offers a clear and easy understanding of the financial situation.

In the end we have the manager who has the access for all these dashboards to confirm their decisions or take the decision in their places.

VI. Conclusion

Overall, the realization chapter demonstrated the successful implementation of our system, leveraging a robust software environment, efficient data modeling, and integration techniques. These efforts contribute to achieving our project's goals of creating an immersive and flawless experience for all the park visitors while ensuring lasting profitability.

General Conclusion

In conclusion, the Amusement Park Dashboard project represents a significant effort to enhance the visitor experience and optimize operational efficiency within the park. By leveraging data-driven insights and innovative technologies such as predictive analytics, real-time monitoring, and personalized recommendations, the dashboard aims to address key challenges faced by theme parks, including long wait times, unpredictable weather conditions, and ensuring high levels of customer satisfaction.

The development of the dashboard involved the creation of a robust data warehouse to store and analyze various park metrics, such as visitor demographics, ride popularity, and revenue generation. Additionally, predictive models and interactive dashboards were designed to provide park managers with actionable insights to improve ride allocation, staffing levels, and marketing strategies.

One of the standout features of the dashboard is its ability to dynamically adjust to changing park conditions and visitor preferences, ensuring a seamless and enjoyable experience for all guests. Moreover, the integration of real-time feedback mechanisms and customer engagement tools allows for continuous improvement and adaptation to evolving visitor needs.

Overall, the Amusement Park Dashboard project has the potential to revolutionize the way theme parks operate and engage with their guests. By embracing technology and data-driven decision-making, parks can enhance customer satisfaction, maximize profitability, and create memorable experiences for visitors of all ages. However, ongoing investment in innovation and sustainability will be crucial to ensuring the long-term success and relevance of such initiatives in the ever-evolving leisure and entertainment industry.